Progress Report

Evaluating the performance of lock-free data structures under realistic exchange workloads

CS310 Third Year Project

5508367 - Louis Tanak

Supervised by Dr. Sara Kalvala

Department of Computer Science

University of Warwick

21 January 2026

1. Introduction

The concept of a lock-free structure was first publicly discussed by Michael Maged in 2007 at the PLDI conference [1]. The lock-free memory allocator introduced was an alternative approach to achieve multi-threading safety when allocating memory, providing benefits such as immunity to deadlock, preemption-tolerance and async-signal-safety [1, p. 35, §1].

The results were significant: the allocator provided a latency speed-up of approximately $2.0972\times$ across three different traditional memory allocators, across multiple tests [1, p. 43, §4.2.1]. Further benchmarks demonstrated even more desirable benefits, such as improved scalability, a 15% contention-free speed-up in uniprocessor systems, and better space efficiency over all its competitors [1, p. 44-45, §4.2]. The paper concludes with the statement that the work done would allow for efficient algorithm implementations, for structures such as LIFO stacks, FIFO queues, linked lists and hash tables. Michael implies that these structures could be completely dynamic and lock-free, without requiring any additional OS support [1, p. 46, §5].

Fast forward over 18 years, lock-free data structures are a well-established area of academic research. Modern research primarily focuses on designing and implementing various lock-free solutions, to maximise throughput, minimise latency and scale efficiently. However, much of the research does not benchmark order-preservation, nor evaluates the structures under realistic workloads.

Lock-free data structures have various real-world applications, most notably within the financial industry. They are common in high-performance trading exchanges, where low-latency and concurrent access is vital. Despite their importance, there is limited publicly available research on how these structures perform within a realistic exchange pipeline, including how the characteristics of specific lock-free implementations affect downstream systems. This high-

lights the need to evaluate lock-free data structures beyond synthetic benchmarks, and in more realistic applied contexts.

Therefore, the project undertaken evaluates the performance of traditional and lock-free solutions under realistic exchange workloads. Data structures are benchmarked under stress and order-preservation testing to investigate metrics such as latency, out-of-order completion, and how ordering could affect the matching of orders within an exchange. The implementation of the project leverages modern `C++ 17/20/23`, to provide a low-latency benchmarking framework and to conform to realistic workloads, resulting in a controlled environment setup that would be able to effectively evaluate the data structures.

The work aims to bridge the gap between the theoretical and synthetic benchmarks commonly found in academic research and the practical use of lock-free data structures in industry. The results would provide insights into the benefits, limitations and the suitable conditions for each data structure, enabling comparison with the traditional synthetic benchmarks, as well as highlighting the real-world implication of the characteristics of the data structure being evaluated.

2. Background & Related Work

2.1 Lock-free Data Structures & Matching Engines

Notable studies in lock-free data structure research evaluate performance and scalability under stress-testing conditions, utilising synthetic benchmarks to measure the throughput, latencies and the behaviour as thread count increases.

The first study by Ruslan Nikolaev introduces a Scalable Circular Queue (SCQ), a design portable across virtually all existing architectures, with exceptional memory efficiency and performance compared to other structures and algorithms [2]. At a high-level, SCQ utilises a fixed-size buffer and separates the logical ordering of elements from their physical memory location[2, p. 3, §3]. The `enqueue` operations acquire a ticket which defines the logical ordering of elements within the queue, preserving FIFO semantics at the abstraction-level of the program. However, since there is a separation between the logical ordering and the physical memory location, it does not guarantee that ordering is preserved in terms of observable execution or completion order[2, p. 13, §7]. For the majority of concurrent applications this is negligible, however systems with strict ordering constraints would be sensitive to this behaviour.

Whilst this study benchmarks performance across a variety of thread counts and different algorithmic contention levels, its focus is on the throughput and latency metrics derived from these synthetic workloads[2, p. 11-14, §7]. Despite FIFO semantics being maintained by design, the paper does not explicitly measure ordering behaviour during execution. Following this, the downstream effects caused by out-of-order completion on dependent systems are not discussed, as they are not within the primary scope of study.

Similarly, in broader lock-free research, it is common for implementations to prioritise performance metrics such as latency, evaluated from synthetic benchmarking. For example, open-source implementations such as the `moodycamel`

`ConcurrentQueue`[3] or the `MPMCQueue` by rigtorp[4], only report the latencies, throughput and memory usage of their structures. Some open-source implementations explicitly state if FIFO or other ordering guarantees are maintained, yet many do not measure this, and multiple do not discuss ordering constraints altogether. This pattern further indicates an evaluation gap in the assessment of order preservation and potential downstream effects caused by this.

Beyond isolated data structure research, industry-specific systems embed lock-free designs within larger processing pipelines. A notable example is the LMAX Disruptor[5], a high-performance, lock-free inter-thread communication framework, developed for the LMAX financial exchange[6]. The Disruptor was designed to minimise coordination overhead between different stages of a processing pipeline, addressing the performance limitations of traditional queue-based designs which were unsuitable for the ultra low-latency requirements of a financial exchange. The paper performs throughput and latency testing of the disruptor in comparison to Doug Lea's `ArrayBlockingQueue`[7], and concludes that the Disruptor is a major step forward for increasing throughput and reducing latency between concurrent execution contexts[5, p. 11, §7].

The LMAX disruptor paper represents one of few publicly available studies that evaluates lock-free mechanisms within financial exchanges, as most matching-engine implementations remain undisclosed due to the nature of the industry. In a similar vein to lock-free research, the paper focuses on the performance metrics of their specialised design, and does not evaluate behavioural aspects such as execution-ordering, memory-usage and its interaction between other components of the pipeline. This results in limited insight into how individual, general-purpose lock-free data structures behave within an exchange-style pipeline.

2.2 Identified Research Gap

In response to the limitations identified in existing literature, this project provides an evaluation of lock-free and traditional data structures within an exchange-style pipeline. The project extends conventional stress-testing to include the analysis of execution behaviour under realistic workloads, enabling the observation of order preservation and its effects on downstream components. This is achieved through pseudo-random transaction order generators, to mimic realistic market patterns found in exchanges, which are then processed through

the data structures. Stress testing would provide insight into latency measurements similar to prior research, whilst order-preservation testing would process orders through a price-time priority matching engine. This would enable the evaluation of how execution behaviour influences trade sequencing and resulting price information. Further to latency and ordering measurements, the system captures memory usage and hardware-level performance characteristics, to provide a stronger understanding of how individual lock-free data structures behave within realistic exchange-like pipelines.

3. Project Progress

This chapter presents the current progress of the project, discussing the overall evaluation framework and key design decisions made to ensure the validity and research suitability of the results. A discussion on the core components of the system in greater detail will follow, motivating the chosen designs and highlighting implementation considerations. Preliminary experimental results are subsequently presented, and the chapter concludes with the limitations and challenges encountered during development.

3.1 Evaluation Framework Design

Figure 3.1 represents the high-level activity diagram for evaluating lock-free data structures. The user provides input to the type of test to conduct, the number of threads and the number of orders to be processed.

3.1.1 Stress Testing

During stress testing, the framework measures the `enqueue` and `dequeue` latencies of the data structure being evaluated. Producer and consumer threads repeatedly `enqueue` and `dequeue` from the data structure until the specified order limit is reached. Listing 1 presents the pseudocode for an `enqueue` thread. Each thread stores a local counter to ensure that it does not exceed its thread order limit, and each thread provides its own pseudo-random order generator to enable repeatability across benchmark runs. The corresponding `dequeue` threads provide similar functionality, without the need for order generation.

Stress testing aligns with evaluations in existing literature and serves as a baseline experiment. It provides a point of reference for validating the latency

measurements produced, to ensure results fall within a reasonable range of reported performance characteristics of lock-free data structures.

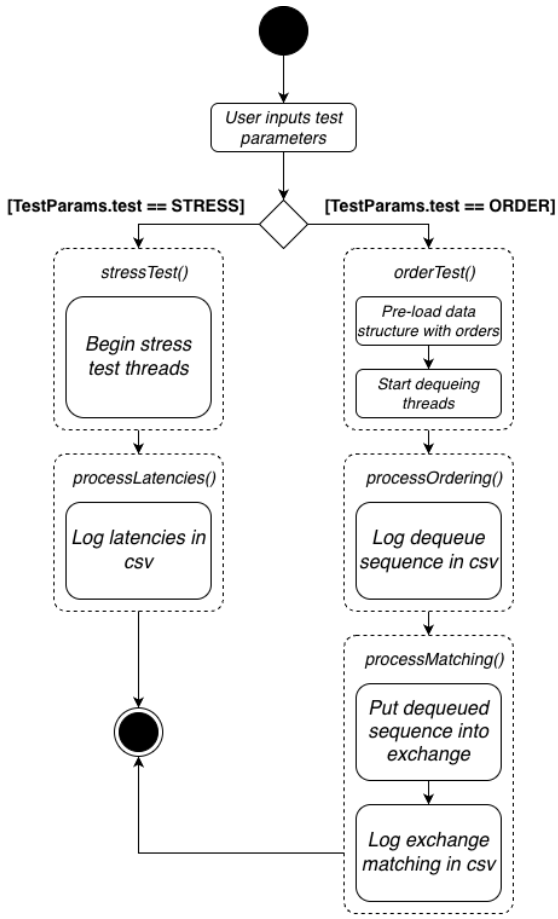


Figure 3.1: *Activity diagram representing the sequence of events for testing lock-free data structures at a high-level.*

```
count = 0;
RandomOrderGenerator gen(seed=0); // seeded order generator
while (true){
    if (count >= THREAD_LIMIT) break;
    // generate an order
    order = gen.generate();
    ++count;
    // enqueue the order into the data structure
    wrapper.enqueueOrder(order, thread_id);
}
```

Listing 1: *Example pseudocode for the* `enqueue` *threads in the stress testing scenario.*

3.1.2 Order-preservation Testing

Order testing is the primary contribution of this project, providing the ability to evaluate order-preservation properties of lock-free data structures and their downstream effects through a simulated trading exchange.

Rather than recording the operation latencies, the `order_id` and the timestamp of the dequeued orders are stored. The collected entries are sorted by timestamp to reconstruct the true dequeue sequence. This reconstructed ordering is used to analyse the extent which the data structure has preserved the original insertion order.

Following this, the downstream effects of the ordering inversions are evaluated. The reconstructed dequeue sequence is simulated through a price-time priority exchange, in order to demonstrate the potential impacts of out-of-order completion within a specific industry application.

Once all trades have been processed, the exchange state is cleared and simulated under the expected ordering. Both expected and observed trade sequences are recorded, providing a direct comparison of the effects of out-of-order completion on trade matching.

Contributing order-preservation testing provides insight into how subtle concurrency effects can propagate into larger downstream effects, causing economically significant changes. This type of evaluation is not commonly addressed in existing literature, therefore offering a novel perspective on the practical implications of lock-free data structures.

3.2 Design Decisions

The project has a strong emphasis on result reproducibility and low-latency design, particularly on testing hot paths. To ensure the validity of results obtained, it is essential to minimise overhead between the operations being benchmarked, especially during stress testing. Further to this, adopting a low-latency approach throughout the framework allows the evaluation to reflect realistic performance characteristics of real-world systems.

3.2.1 Low-latency Timestamping

One subtle yet critical aspect to consider is the timestamping mechanism. During stress testing, timestamps are recorded before and after each operation, and the difference is the elapsed time for the operation. However, timestamping itself induces its own overhead, and commonly used solutions may be unsuitable for benchmarking lock-free data structures, where operation latencies are typically a few nanoseconds.

For example, `std::chrono::high_resolution_clock` [8] provides a convenient way to achieve timestamping in `C++`. However, benchmarks report an average latency of approximately 53.05*ns* [9, Example 9], which does not meet the low-latency requirements of this project.

To minimise measurement overhead, this project uses the `RDTSC` instruction to read the CPU timestamp counter. Previous benchmarks demonstrate that `RDTSC`-based timestamping takes approximately 7.6*ns* (18 clock cycles)[10], making it more suitable for low-latency benchmarking in comparison to high resolution clock. The exact timestamping implementation used can be seen in Listing 2.

```
// function inlined for further latency improvements
inline auto rdtsc_lfence() -> uint64_t {
    uint32_t lo, hi, aux;
    asm volatile("lfence; rdtsc" : "=a"(lo), "=d"(hi) :: "memory");
    return (hi << 32) | lo;
}
```

Listing 2: *`rdtsc_lfence` function for efficient timestamping.*

3.2.2 Templated Programming

Templates in `C++` enable the creation of generic classes or functions in the language[11]. This allows a single piece of code to be reused for multiple data types, without requiring multiple implementations. For a benchmarking framework, this is extremely powerful as it allows benchmarking functions to be applied across different lock-free data structure implementations.

An alternative design approach is to utilise inheritance and virtual functions. A base class would define an abstract interface, with concrete data structure im-

plementations overriding the base methods using virtual methods to provide the data-structure specific behaviour, via run-time polymorphism.

Despite both approaches being viable, templates are preferred in this project as virtual functions provide an additional run-time overhead[12]. This is due to the need to look up the address of the function at runtime, going through the virtual function table. In contrast, templates are compile-time mechanisms. The compiler generates specific functions for each concrete data type that the template uses within the program, resulting in distinct functions. This comes at a cost of increased compile time, yet these trade-offs are acceptable for this project as they do not impact performance. The project leverages templated-programming wherever possible, to minimise overhead whilst maintaining portability and code reuse.

3.2.3 Pseudo-random Testing

To achieve consistency and reproducibility amongst benchmarking experiments, the project utilises pseudo-random (commonly known as seeded) order generation. A seed is used to produce a deterministic sequence of numbers that appear random, providing a means to have tests replayed across multiple different benchmark executions. This supports isolating individual characteristics of lock-free data structures being tested, and permits for more thorough testing.

Within the project, order generators accept an optional seed parameter, providing a controllable mechanism for testing. These have been implemented using the `std::mt19937` class[13], which produces 32-bit pseudo-random values using the Mersenne twister algorithm[14].

In the `RandomOrderGenerator` class, attributes such as price, quantity, and order type are derived from the pseudo-random values and normal distributions. Values are sampled by parameterising the normal distribution, to enable the generation of values within the pre-defined bounds. An example of this can be seen in Listing 3.

```
// within constructor
std::mt19937 rng_ = std::mt19937(seed.value());
...
// within generateOrder();
```

```
std::normal_distribution<double> priceDist(base, base * vol);
double price = priceDist(rng_);
```

Listing 3: *Code for seeded order generation, demonstrating how* `price` *is sampled using an input seed.*

3.3 Benchmarking Class Design

The `BenchmarkWrapper` class is the primary abstraction in which the different benchmarking scenarios interact with the underlying lock-free data structures. A subset of the methods and attributes can be seen in Figure 3.2. The class is templated on the data structure implementation and order type, shown by the «$X \rightarrow Y$» notation. This allows for different lock-free data structures to be substituted and consequently evaluated, without modifying the class itself.

Figure 3.2 demonstrates the interaction between the class and a concrete data structure. A generic queue interface `IQueue` is bound at compile-time to the concrete class `MCLockFreeQueue`, which in this scenario implements the `moodycamel` `ReaderWriterQueue`. As template bindings are resolved at compile-time, it avoids the runtime overhead discussed in section 3.2.2. This design demonstrates the simplicity behind substituting alternative data structure implementations and ensures there is no additional bias from the framework, as the class only interacts with the templated interface for the data structure, and not the structure itself.

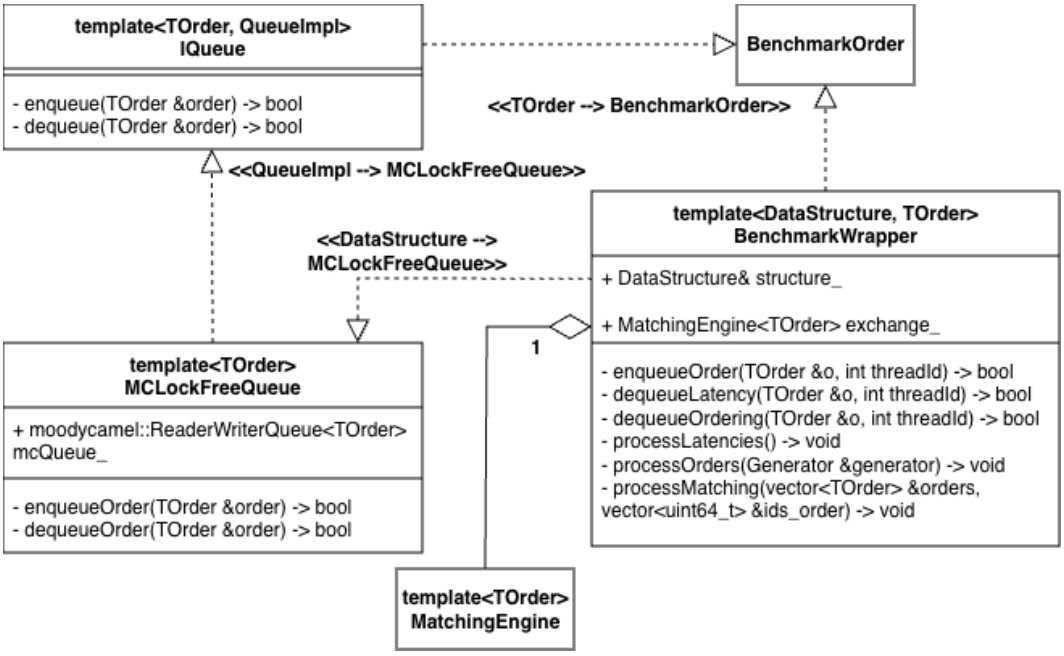


Figure 3.2: *UML diagram for* `BenchmarkWrapper` *displaying a subset of the attributes and relationships.*

The wrapper exposes `enqueue` and `dequeue` methods directly to the producer and consumer threads that manipulate the data structure within testing scenarios. These methods act as lightweight forwarding layers to the underlying data structure, whilst performing the necessary measurements required for further analysis after the test has been completed. Measurements collected during the execution are stored internally in the wrapper. This ensures that there is no cross-component synchronisation for collecting measurements, simplifying the post-processing of the data collected.

Overall, `BenchmarkWrapper` acts as a coordination layer for the framework, providing access to the data structure, storing collected measurements and exposing the post-processing methods all within a single class. A template-based design ensures the class remains extensible and suitable for evaluating a wide-range of lock-free data structures in a consistent manner.

3.4 Orderbook & Matching Engine Design

The complete UML for the matching engine is shown in Figure 3.3, generated by `clang-uml` and displayed with `PlantUML`. To highlight the most relevant parts, Figure 3.4 will be referenced.

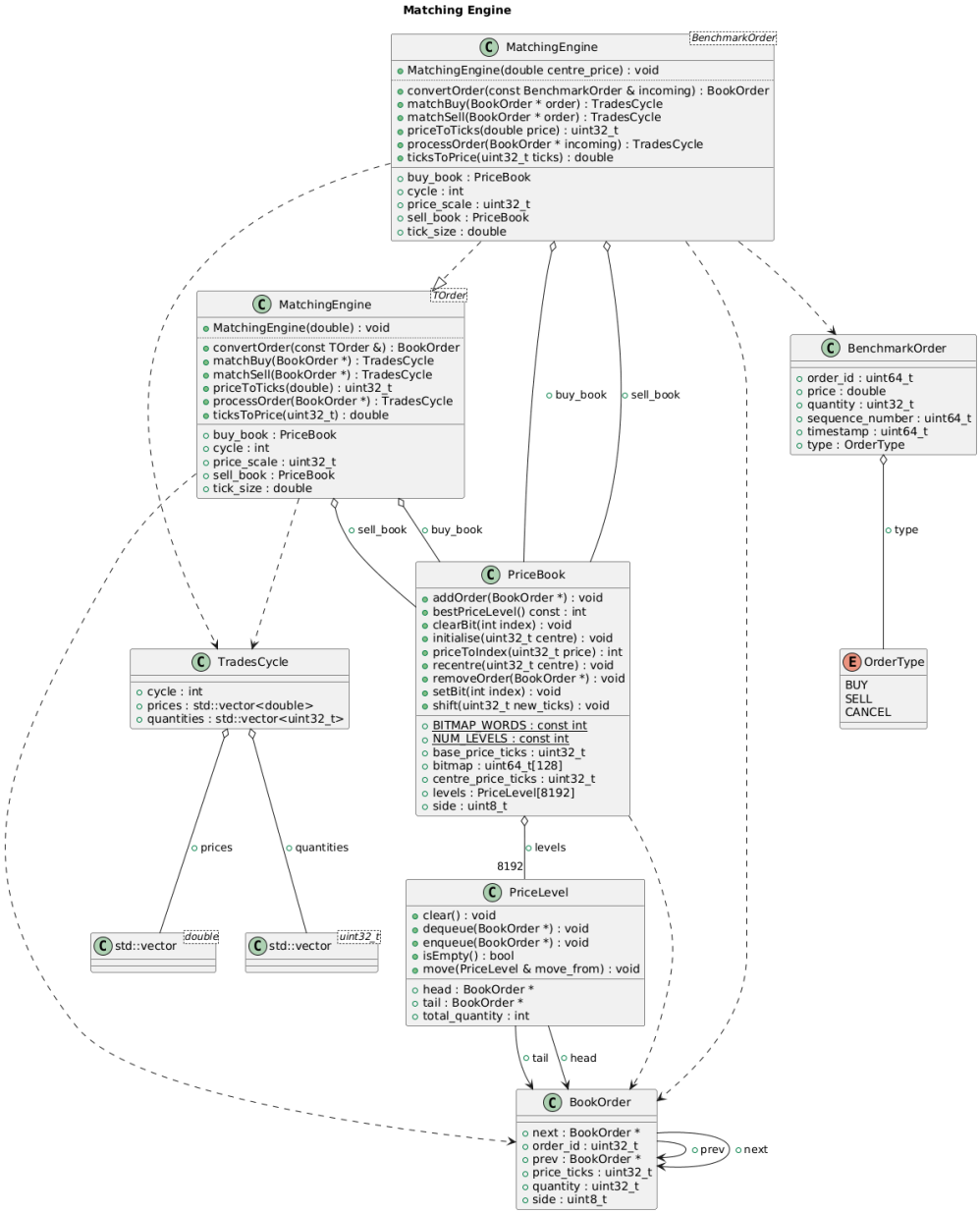


Figure 3.3: *UML diagram for the entire exchange workflow.*

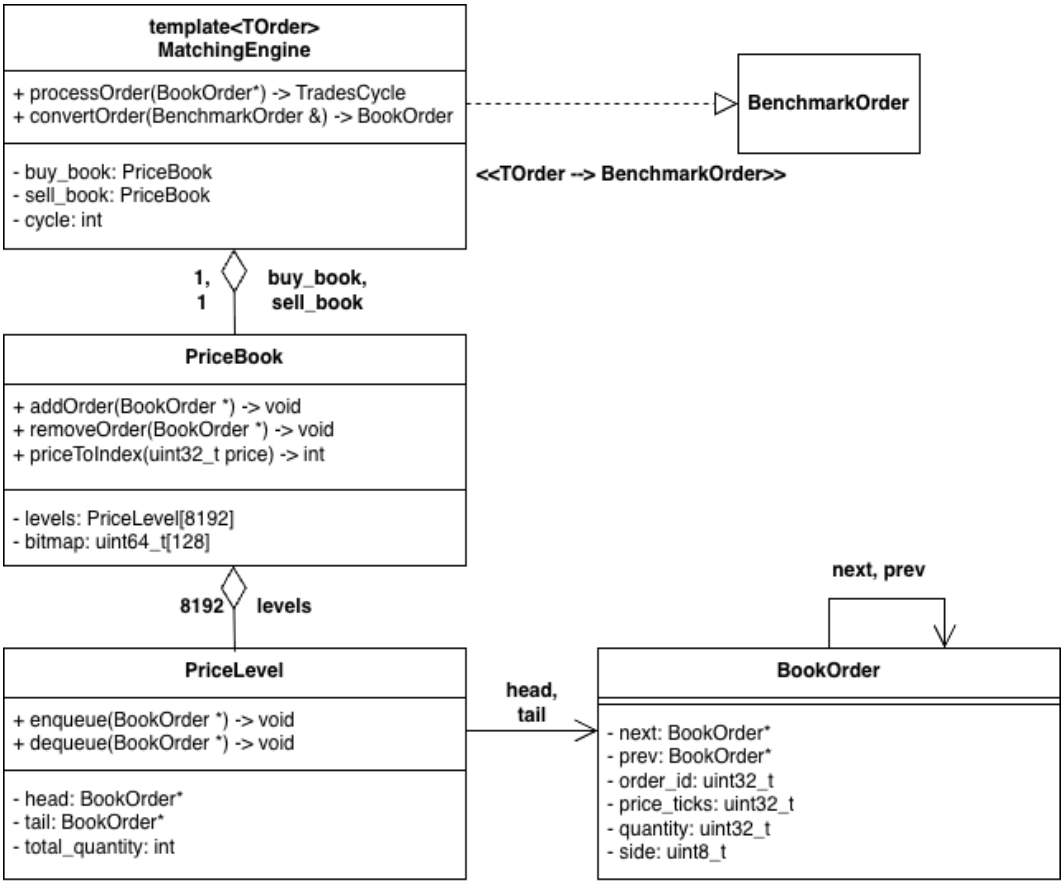


Figure 3.4: *UML diagram illustrating a subset of methods and attributes in the exchange workflow.*

Although the exchange operates within the post-processing pipeline, it adheres to low-latency design considerations. Post-processing may involve a large volumes of data, therefore an exchange with efficient core-operations will post-process data significantly faster than an inefficient implementation, saving time during the results analysis process.

At the top-level, `MatchingEngine` maintains separate order books for buy and sell orders. A match occurs when the best price of a buy order exceeds the price of the cheapest sell order. Each book stores a `PriceLevel`, where each level represents an individual price, for example, 123.45 and 123.46 are adjacent price levels. Each level stores a linked list of orders, where when an order is added to a price level, it is added to the tail of the linked list, to sort them by timestamp.

We use a bitmap to indicate which price levels are currently active. Finding the best price has a time complexity of $O(1)$, utilising the code seen in Figure 4.

This is because `BITMAP_WORDS` has a constant size of 128, as we store 128, 64-bit numbers to express all 8192 different indexes in the `levels` array. Therefore, the loop only runs at most 128 iterations. The instruction `__builtin_ctzll` is a single CPU instruction on modern architectures, therefore the overall complexity is $O(128) \equiv O(1)$. Array index lookup is known to be $O(1)$ and inserting an order into the linked list of a price level is also $O(1)$, as we store a reference to the last element in the linked list.

```
auto PriceBook::bestPriceLevel() const -> int {
// With 8192 levels, we store 128, 64-bit values
// Each individual bit represents an index
    for (int word = 0; word < BITMAP_WORDS; ++word){
        if (bitmap[word]){
            return (word * 64) + __builtin_ctzll(bitmap[word]);
        }
    }
    return -1;
}
```

Listing 4: *`bestPriceLevel()` function, which finds the index of the best price level in $O(1)$ time complexity.*

This exchange design was utilised as it provides a strong balance between simplistic implementation and strong performance, and is sufficient in simulating the results of order matching in an exchange.

3.5 Preliminary Experiment Results

These preliminary results are obtained from benchmarking the `moodycamel ConcurrentQueue`, whilst the final report will evaluate a wider range of lock-free data structures.

Regarding stress testing, the current framework implementation closely resembles the latencies in the published benchmarks [15]. The reference benchmarks report latencies in the range $[22.9ns, 23.0ns]$, whilst the framework produced results in the range $[39.9976ns, 59.9964ns]$, seen by Figure 3.5. Given hardware differences and the remaining limitations (discussed in §3.6), these are reasonable results to have been achieved at this stage.

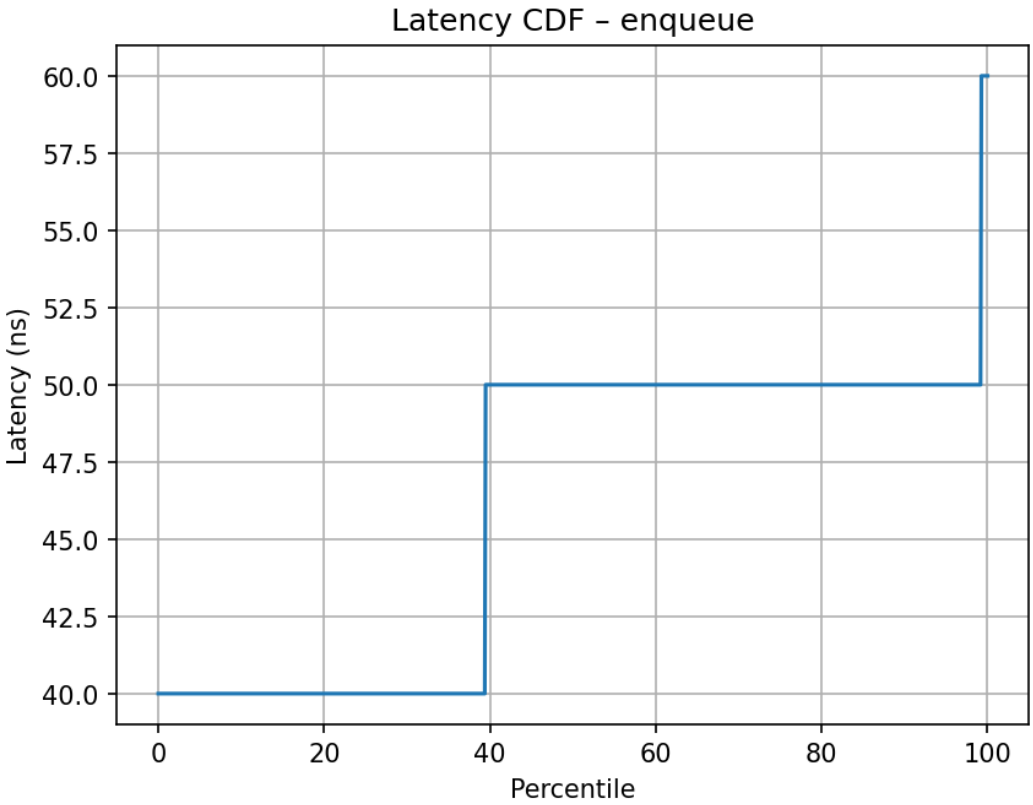


Figure 3.5: *Latency against percentile for a single-producer, single-consumer test.*

Order-preservation testing provided particularly interesting results. The `moodycamel` queue does not preserve strict ordering, seen in Figure 3.6. Certain orders are processed later than expected, resulting in negative offsets. Across 40,000 orders, with four consumer threads, we can see offsets of up to approximately 350 positions, a significant difference that could demonstrate a large effect on downstream systems.

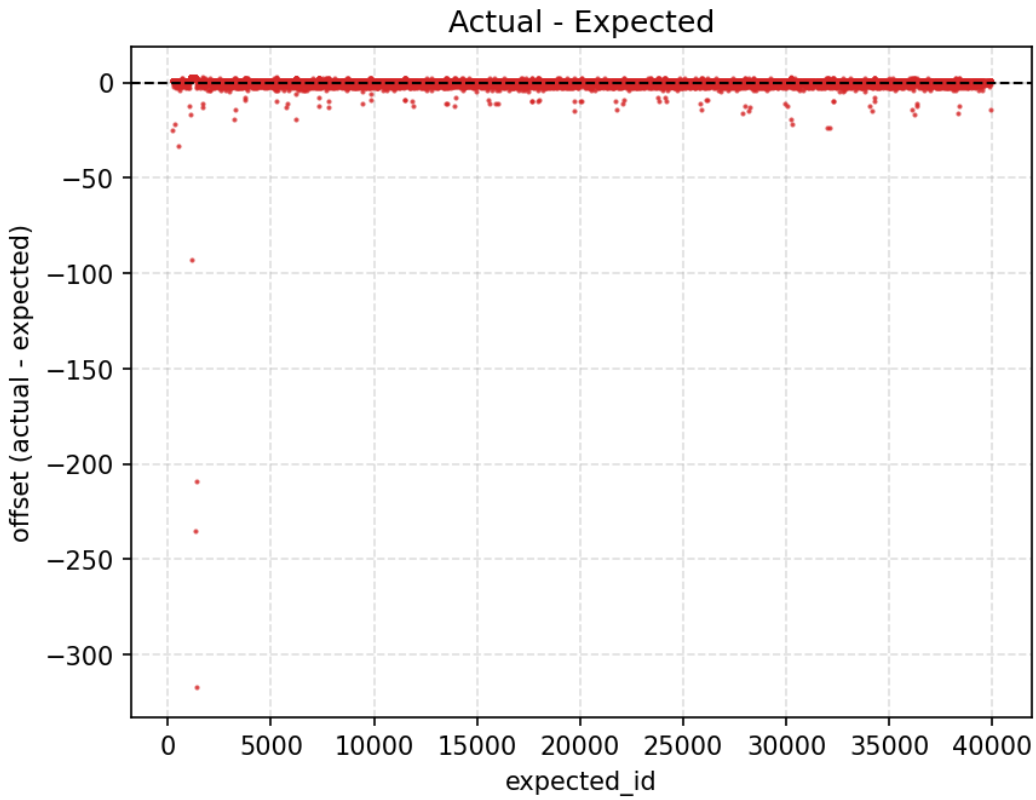


Figure 3.6: *Offset between the expected and actual order IDs. Results show out-of-order completion.*

To identify finer trends, Figure 3.7 shows the first 1,000 orders processed. Around order ID 210, the expected order is not dequeued, and 25 orders are executed serially, each offset by one. Eventually, the original order is dequeued 25 positions after expected. Similar patterns occur at IDs of approximately 310 and 480, indicating this is a characteristic of this specific data structure. Symmetry across the offset line indicates out-of-order pairs, frequently seen throughout but clearly seen at IDs 250, 600 and 800. Overall, 7,128 orders of 40,000 were mismatched, resulting in a 17.82% mismatch percentage.

These results demonstrate an importance for order-preservation testing. Systems requiring a strict sequence of orders would be heavily affected by implementing a lock-free data structure such as the one tested, with potentially substantial downstream effects.

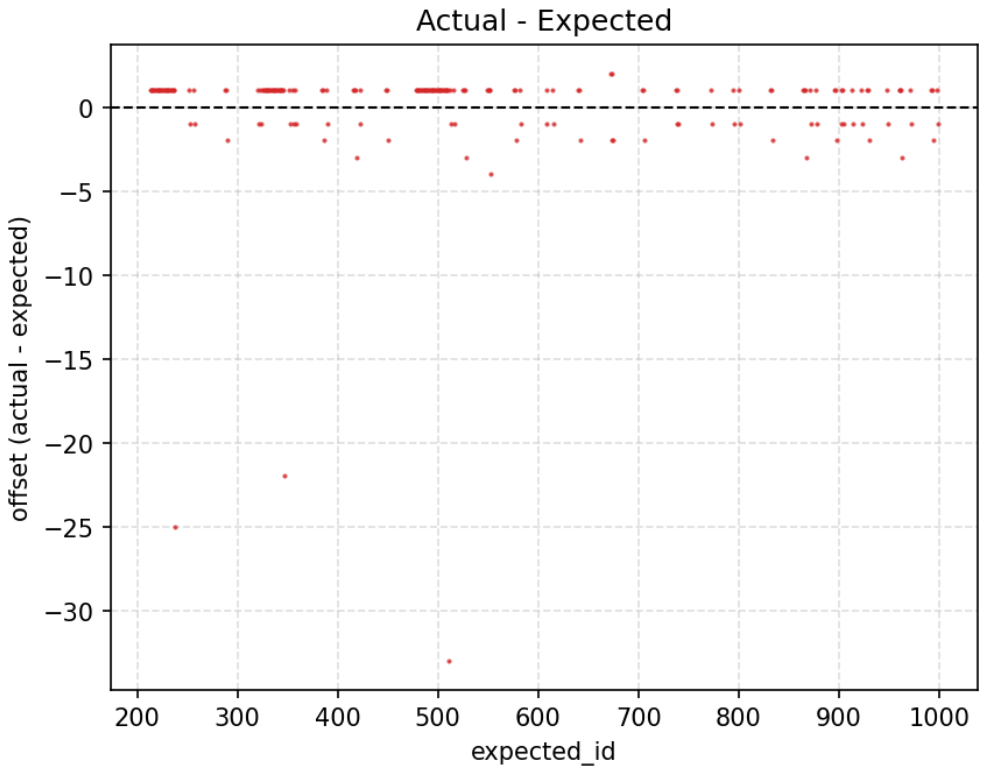


Figure 3.7: *Offset between the expected and actual order IDs for the first* $1,000$ *orders.*

The downstream impact of order-preservation on an exchange are shown by Figures 3.8 & 3.9. Figure 3.8 shows the expected candlestick chart for the previously used interval. Figure 3.9 overlays the actual matching outcome with transparent candlesticks.

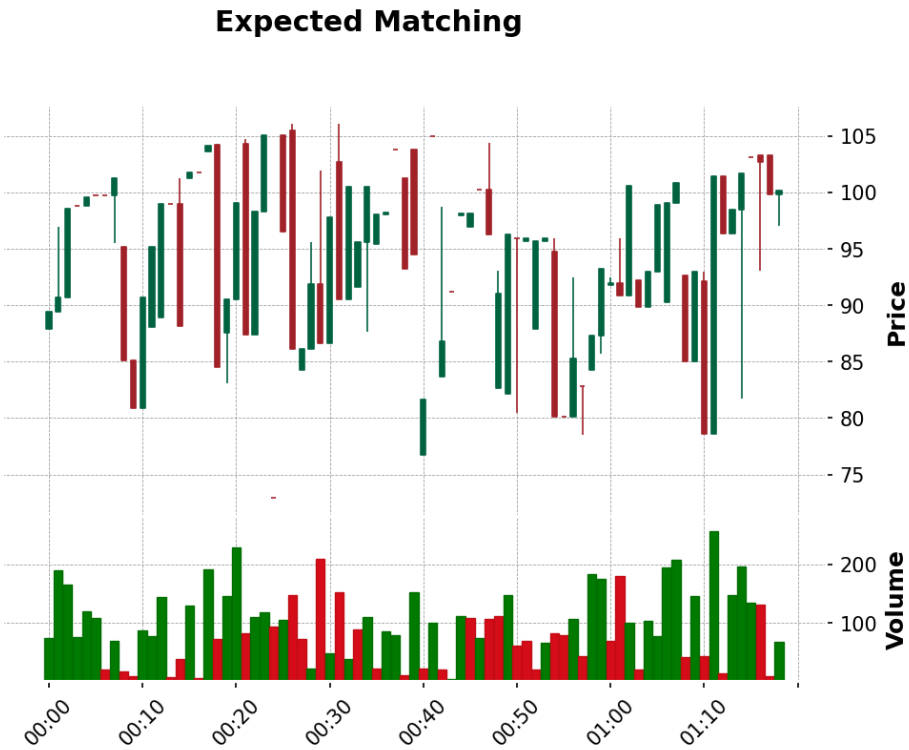


Figure 3.8: *Expected exchange outcome, represented as a candlestick chart.*

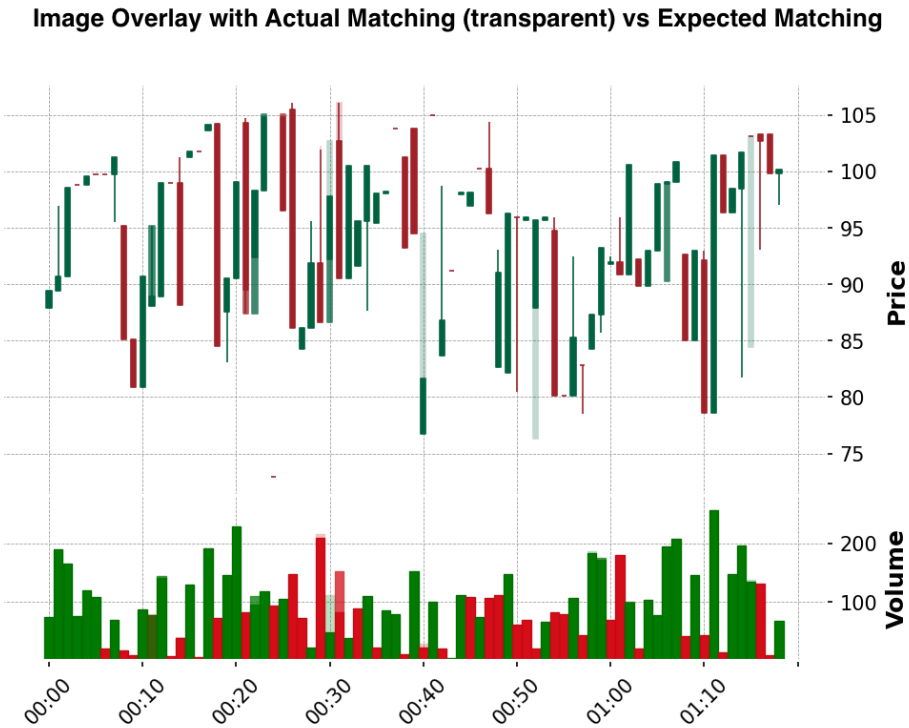


Figure 3.9: *Overlay with actual exchange outcome as transparent candlesticks.*

The results clearly demonstrate differences in exchange behaviour. There are significant differences in open and close positions at `00:40` and `01:20` on the diagram. Traded volume also changes significantly, clearly seen at `00:30`, where the notional volume is approximately double in one cycle, then halved in the next.

These findings reinforce the conclusion that ordering inversions within lock-free data structures could have meaningful economic impact. Although lock-free designs are widely used in exchanges, insufficient order-preservation can lead to unintended unfair outcomes, differing price information between exchanges and arbitrage opportunities, exploiting market inefficiencies.

With these preliminary results, it will be extremely interesting to investigate these metrics further in the final report, and provide a stronger analysis to support these conclusions.

3.6 Identified Limitations & Challenges

A limitation of the current solution is the potential for contention and cache inefficiencies when benchmarking concurrent lock-free data structures. When a thread records the latency of an operation, it writes to a shared latency buffer within its own assigned slice. Despite this design avoiding write contention between threads, it causes frequent cache line evictions. Slices are spaced apart in memory, meaning each write may require loading a different cache line, increasing cache miss rates, increasing latency. This is evidenced through preliminary latency testing, where the same data structure under single-threaded execution averaged a latency of $123.866ns$, whilst with four threads the latency was $452.454ns$. To mitigate this, using per-thread buffers that are explicitly aligned in cache would reduce interference between threads and may improve the cache miss rate. This work is planned to be completed during the remainder of the project.

A further limitation is the absence of pre-processing techniques or cache warming prior to measurement. The first `enqueue` and `dequeue` operations typically take the longest due to cold caches. One example tested had the first `enqueue` operation take $96824.1ns$, and the equivalent `dequeue` operation taking $1159.93ns$. Including these outliers skew the average result calculations, providing misleading information whilst benchmarking, reducing result validity. To address this, cache-warming and other pre-processing techniques will

be implemented to better reflect steady-state conditions, providing more representative latency testing measurements.

These issues will be addressed to provide a comprehensive framework that is suitable for both latency and order-preservation testing.

4. Planned Work & Appraisal

4.1 Critical Appraisal of Current Progress

Overall progress on the project has been consistent and aligns with the original objectives defined. Initial results demonstrate that the framework can produce viable research outputs, specifically the order-preservation testing providing a novel contribution to analyse the downstream effects of ordering inversions. Currently, the project is on track to complete all the core objectives defined in the original specification.

Despite steady and consistent progress, external factors have hindered progress at some points. For example, a competing coursework deadline was not included in the original timetable, which required immediate attention. This highlights the importance for more thorough planning and contingency in the schedule. However, this disruption was temporary and did not impact the scope of the project.

4.2 Remaining Implementation

The original timetable and objectives remain unchanged, as progress has been made to account for the delays without compromising the outcomes. The remaining implementation follows on from Objective 6, where more realistic order generation will be implemented.

5. Project Management

5.1 Development Methodology

The original software development methodology remains unchanged from the project specification. The project continues to use an agile methodology, to incrementally build up the solution with each iteration. To explicitly achieve incremental development, the project objectives and the Gantt chart are written where each iteration adds functionality in a pre-determined time-frame, aligning with the official PMBOK®Guide [16, p. 50, §1.2.4.1]. The agile methodology interleaves the design, research and implementation of a certain feature, ensuring any prerequisites to implementing code is covered before undertaking the development of a feature.

To ensure each section of the project functions as expected, unit, component and integration tests have been implemented. These have been implemented via GoogleTest [17], GitHub Actions [18] and shell scripts. GoogleTest is a specialised library tailored to testing in `C++`. GoogleTest provides a large set of features to the developer, and provides simple syntax for developing tests as shown by Listing 5, where the developer defines the category and test name and performs assertions through the scope of the test.

```
TEST(TimingTest, MeasureTscGhzReasonableRange) {
        double ghz = ltime::measure_tsc_ghz();
        EXPECT_GT(ghz, 0.1);   // GHz must be positive
        EXPECT_LT(ghz, 10.0);  // typical CPU upper bound guard
}

```

Listing 5: *Example unit test implemented to test CPU clock-speed measurement.*

Shell scripts enable testing during development, whilst GitHub Actions ensures all tests pass before allowing a pull-request to be approved. GitHub

Actions is used to run the testing functions, and permits merge commits if all test cases pass, as seen by Figure 5.1.

Testing has ensured the reliability of the project throughout its development, with new functionality validated upon completion. Automated testing reduced the risk of erroneous code and helped maintain progress in line with the initial project specification.

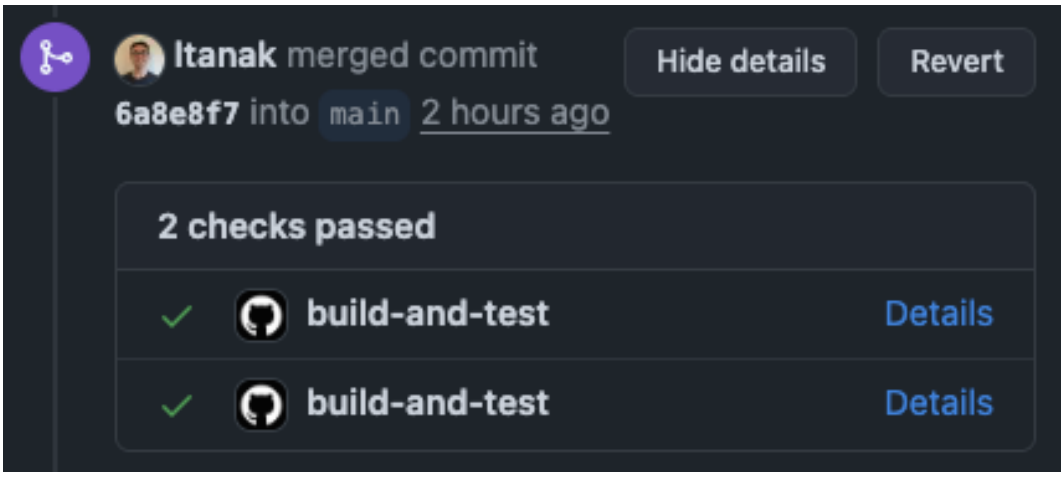


Figure 5.1: *Screenshot of tests passing, enabling for commit to be merged onto the* `main` *branch.*

5.2 Progress Monitoring

Progress has been monitored using a Kanban board on Trello, seen by Figure 5.2. The Kanban board highlights the time frame for work to be completed and aligns each task with its corresponding project specification objective, remaining consistent with the original Gantt Chart. Kanban boards are highly valuable for comparing actual progress against expected performance, and are recommended by official project management literature [19, p. 109, §2.7.3.3]. In this project, the board has been an effective tool for monitoring progress and will continue to be used throughout the remainder of the work.

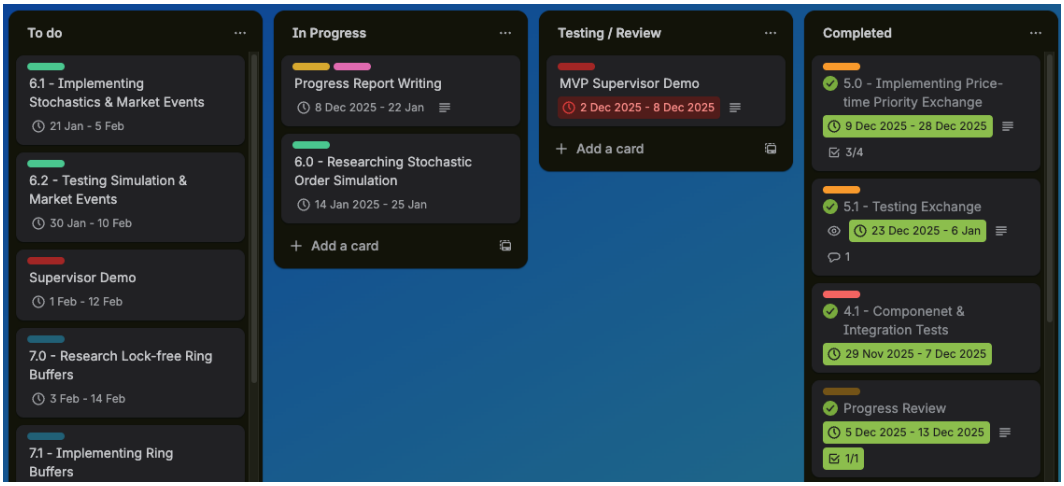


Figure 5.2: *Snippet of Trello board used to monitor progress.*

Alongside the Kanban board, daily project activities were planned using a personal calendar, accounting for lectures, seminars and other university work. This was to ensure that sufficient time was allocated to the project each week, and has proven valuable for maintaining steady progress. Figure 5.3 illustrates an example of how daily work would be planned.

Figure 5.3: *Personal calendar demonstrating planning of dissertation work.*

5.3 Comparison Against Initial Project Specification

Compared with the original timeline proposed, the project is on track to be completed within the allocated time-frame. Whilst progress did fluctuate throughout the development period, with some features completed ahead of schedule and others temporarily delayed, these were addressed through improved time management of the project. Additionally, the iterative development approach of the project enabled for more accurate estimation of feature complexity and duration, allowing for the updated Gantt chart to reflect these time-frame changes, seen in § 4.2.

Overall, the project continues to align with the original specification. Essential work has been completed, and the remaining core objectives are well defined and scheduled, providing confidence that the project and its outcomes will be achieved in the remainder time-frame.

6. Legal, Social, Ethical and Professional Issues & Considerations

The professional and ethical considerations from the initial project specification remain unchanged, with respect to proprietary exchange information and the potential concern of personally identifiable information in datasets.

7. Bibliography

[1] M. M. Michael, "Scalable lock-free dynamic memory allocation," in *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI '04)*. ACM Press, 2004. doi: 10.1145/996841.996848. ISBN 1-58113-807-5 pp. 35–46. [Online]. Available: https://www.cs.tufts.edu/~nr/cs257/archive/neal-glew/mcrt/Non-blocking%20data%20structures/p35-michael.pdf

[2] R. Nikolaev, "A scalable, portable, and memory-efficient lock-free FIFO queue," *CoRR*, vol. abs/1908.04511, 2019. [Online]. Available: https://arxiv.org/pdf/1908.04511

[3] D. Cameron, "moodycamel::concurrentqueue: A fast multi-producer multi-consumer queue for c++," https://github.com/cameron314/concurrentqueue, 2014.

[4] E. Rigtorp, "MPMCQueue: A bounded multi-producer multi-consumer lock-free queue," https://github.com/rigtorp/MPMCQueue, 2016.

[5] M. T. Oliveira, D. Barker, G. Heller, J. Meszaros, and C. Buss, "Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads," LMAX Exchange, Technical Report Disruptor-1.0, 2011. [Online]. Available: https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf

[6] LMAX Exchange, "Lmax exchange – institutional exchange for global fx," https://www.lmax.com/exchange, 2026.

[7] Oracle & OpenJDK contributors, "Arrayblockingqueue.java (java standard library concurrent queue implementation)," https://github.com/JetBrains/jdk8u_jdk/blob/master/src/share/classes/java/util/concurrent/ArrayBlockingQueue.java, 2026.

[8] cppreference.com, "High resolution clock (c++ reference)," https://en.cppreference.com/w/cpp/chrono/high_resolution_clock, 2026.

[9] chronoxor, "Cppbenchmark: A performance benchmark framework for c++," https://github.com/chronoxor/CppBenchmark, 2026.

[10] testing-laboratory, "rdtsctest: A small project for measuring the overhead of the rdtsc instruction," https://github.com/testing-laboratory/rdtscTest, 2026.

[11] GeeksforGeeks. (2026) Templates in c++. [Online]. Available: https://www.geeksforgeeks.org/cpp/templates-cpp/

[12] N. C. Lab. (2024, Nov) Virtual function costs. [Online]. Available: https://nimrod.blog/posts/virtual-function-costs/

[13] cplusplus.com. (2025) std::mt19937 — c++ reference. [Online]. Available: https://cplusplus.com/reference/random/mt19937/

[14] Wikipedia. (2026) Mersenne twister. [Online]. Available: https://en.wikipedia.org/wiki/Mersenne_Twister

[15] C. Desrochers, "A fast general purpose lock-free queue for c++," Online, Nov. 2014, posted 06 November 2014. [Online]. Available: https://moodycamel.com/blog/2014/a-fast-general-purpose-lock-free-queue-for-c++#benchmarks

[16] P. M. Institute, *A Guide to the Project Management Body of Knowledge (PMBOK®Guide) – Sixth Edition*. Newtown Square, PA: Project Management Institute, 2017.

[17] GoogleTest contributors, "Googletest: Google's c++ testing and mocking framework," https://github.com/google/googletest, 2026.

[18] GitHub, "Github actions: Automate your workflow from idea to production," https://github.com/features/actions, 2026.

[19] P. M. Institute, *A Guide to the Project Management Body of Knowledge (PMBOK®Guide) – Seventh Edition*. Newtown Square, PA: Project Management Institute, 2021. [Online]. Available: https://tegnum.edu.pe/wp-content/uploads/2023/09/Project-Management-Institute-A-Guide-to-the-Project-Management-Body-of-

Knowledge-PMBOK-R-Guide-PMBOK%C2%AE%EF%B8%8F-Guide-Project-Management-Institute-2021.pdf

CS310 Project Specification

5508367 - Louis Tanak

2025-10-13

Title

Evaluating the performance of lock-free data structures under realistic exchange workloads.

Abstract

Lock-free data structures allow multiple threads to operate safely on shared data without using locks. These have many real-word use cases in high-performance computing, low-latency systems and financial exchanges. Research has been conducted on their theoretical properties, however there is a gap in understanding how lock-free data structures perform under realistic workloads, such as trading exchange-like patterns.

The work aims to implement various lock-free data structures and evaluate their performance against each other and traditional lock-based solutions. The data structures will be benchmarked against simulated exchange traffic to measure throughput, latency, scalability and completion ordering. Results will provide insights into the benefits, drawbacks and the suitable conditions for each data structure.

1 Problem Statement

In 2007, Michael Maged presented the world's first lock-free memory allocator at the PLDI conference (Programming Language Design and Implementation)[3]. Since then, lock-free data structures have been widely used in a range of applications, across multiple industries.

Previous research has been conducted on the theoretical properties of lock-free data structures, and various studies have analysed the performance of the structures under different load testing, such as increasing thread count or increasing the critical work that each thread does.[1][5]

However, there is limited publicly available work investigating the performance of lock-free data structures with domain-specific implementations under realistic workloads. More specifically, little work has examined their performance under bursty and inconsistent flows, similar to those found in financial exchanges. This provides a gap in the research that this project plans to investigate.

2 Objectives

The primary objectives have been outlined below, separating into the essential and extended work.

2.1 Essential Work

The following tasks are essential work that should be completed to consider the project a success. These provide the foundation for the analysis, benchmarking and experimentation of the project.

1. Design and implement code to simulate trade orders such as buy, sell and cancel orders. These are common in a financial exchange and would establish a realistic workload.

2. Implement different implementations of lock-free data structures. Each data structure will be compared to their locked data structure equivalent, to provide a baseline comparison. Sample data structures include a regular queue, a lock-free queue, and an MPMC queue data structure[4].

3. Design and implement a benchmarking framework that will capture the throughput, latency and the *tail latency* of the data structure being tested.

4. Automate the execution and collection of data from benchmarking by creating configurable test scripts. Automating the tests reduces the probability of human error and provides an environment for controlled testing.

2.2 Core Work

The following is work that should be completed to make the overall project's research value stronger.

5. Design and implement a *price-time priority exchange* to simulate the matching behaviour between orders. It will enable the measurement of additional metrics such as order completion time and fairness, bridging the gap between performance and real-world system behaviour.

6. Introduce stochastic behaviour into the order generation model, such as poisson distributions and simulated *market events*, to create more realistic trading flows to further test data structures.

7. Implement variants of lock-free ring buffers. These are structures used actively in exchanges around the world, making the overall project more realistic and offering an insight into their trade-offs.

8. Investigate methods to ensure correct ordering of transactions and the effect of lock-free data structures on ordering. The benchmarking library will provide information such as the amount of out-of-order pairs and the *spearman correlation* between different data structures.

9. Benchmark hardware-level metrics such as CPU utilisation, cache performance and thread efficiency. Hardware-level metrics provide a deeper understanding of where the differences amongst data structures arise.

2.3 Extended Work

Given enough time, this is work that could be completed, however it is not necessary for its completion.

10. Optimise the data structures for *NUMA architectures*. This would be achieved by binding threads to the same NUMA node, which reduces latency as cores access local memory rather than going through shared interconnection channels. NUMA-aware designs reflect real high-performance exchanges and would lead to substantial latency improvements[2].

11. Optimise each data structure by restructuring memory layout, by using padding and other alignment techniques. The result of this would improve the throughput of the data structure.

12. Design and implement new variants of lock-free queues or ring buffers.

13. Implement one or more data structures in another programming language such as `Go` or `Rust`. This would provide insight into the different memory models of different languages and allow for cross-language data structure comparison.

3 Methods & Methodology

3.1 Pre-reading & Project Setup

At the beginning of this phase, some time will be spent reading `A tour of C++`[6] to understand the syntax and the best practices of the C++ language.

Alongside the reading, to ensure the project is modular and extensible, code will be implemented to create an interface-based architecture, to allow for the different modules to communicate with each other. This will be done utilising templates in `C++`.

Templates allow a developer to write generic code that works with multiple data type or implementation. Another approach that has been considered is utilising *virtual interfaces*, however virtual interfaces are determined at runtime, which would lead to a higher latency. Therefore, templates will be used as they are determined at compile-time and they provide additional type-safety.

3.2 Order Simulation

To simulate transactions, research will be conducted alongside development investigating different methods for generating realistic order flow. This will involve reviewing existing models that simulate markets, including how to accurately simulate market events such as bursty-traffic or large volume transactions occurring.

Once this research has been completed, the logic for this will be completed in `C++`. Multiple producer threads will be creating orders in different patterns, such as following poisson distributions, pseudo-random orders and other possible strategies / distributions found during the research phase.

The orders will be connected directly to the lock-free data structures, for example pushing directly onto the lock-free queue. This provides a direct medium of access to the data structure. Another approach considered was a networked approach, however this creates an additional overhead of variable transmission delays, potential packet loss and would make it more difficult to isolate and measure the true performance of the lock-free data structure being benchmarked.

3.3 Implementation of Lock-free Data Structures

Using the template architecture aforementioned, each individual data structure will be implemented as its own class and conform to the same interface pattern using the template. All the structures implemented will expose the same function signatures to allow them to be used interchangeably as template parameters within the ecosystem of the code. Each data structure acts as a drop-in replacement for one another because they all adhere to the common interface, simplifying the benchmarking process to compare the different implementations.

3.4 Evaluation Methodology

To evaluate the results produced, we will adopt a scientific experimental approach. Benchmarks will be repeated multiple times under identical conditions, and results will be analysed using statistical methods to assess their variability. Following a strict control over the tests will form a more comprehensive analysis, accurately reflecting the behaviour of each data structure being modelled.

3.5 Software Development Methodology

The project will use an agile methodology, to incrementally build up the solution with each iteration. Early iterations will focus on integrating core components of the system, such as implementing a simple order creation model, working with a regular queue implementation and measuring a small amount of metrics with the benchmarking library. Later iterations will result in more comprehensive products, such as more realistic order creations, different variations of the lock-free data structures and potentially passing through a benchmarking library and a simulated exchange simultaneously.

An agile methodology for this project is the most relevant, as it aligns with the experimental nature of evaluating the lock-free data structures. It provides flexibility which will help manage the project within the time constraints, and adjust objectives based on the current stage of development as required.

The code will be regularly tested through a combination of unit, component and *integration tests*. Writing tests ensures that each section of the project functions as expected, whether that is in isolation or as a wider part of the system. The tests will maintain the reliability of the code and promote well-structured and maintainable code.

4 Timetable

The timetable is depicted as a Gantt chart for the proposed project. It includes information regarding project deliverables & deadlines, internal deadlines for code completion or completed components, and accommodates for commitments external to the project.

The scope of the project may change due to strong progress or setbacks. The aim for the first term is to create a minimum viable product, to provide a foundation for implementing and benchmarking the lock-free data structures. The aim for the winter holidays and the second term is to build upon the foundational work and achieve all the core objectives.

The timetable can be found at the end of this project specification.

5 Resources & Risks

5.1 Resources

Technical implementations of the project will be implemented in `C++ 17/20/23`. `C++` is the most suitable language for the project as it natively provides features that would be useful during development. The language includes access to atomic operations, the ability to tune cache-alignment, memory control and more. The majority of real systems are written in the C++ language, therefore it makes it the most suitable language to use, in comparison to other object-oriented programming languages.

Other notable software and technologies that could be used are:

- **Git** - a distributed version control software system that is capable of managing versions of source code or data
- **CMake** - a cross-platform build system generator. It provides a way to unify builds on different platforms such as Windows, Linux and Mac OS
- **perf** - linux tool for performance analysis. Supports subcommands such as hardware performance counters, tracepoints, monitoring events, etc
- **Intel VTune** - profiler which collects performance statistics for serial and multithreaded code
- **Valgrind** - framework that includes various features to help with memory & thread management
- **Python** - high-level programming language with library implementations useful for graphing data
- **Google Benchmark** - library to benchmark code snippets in C++
- **Docker** - containerisation software that packages code with all its necessary dependencies and configuration

These resources may change over the duration of the project.

5.2 Risks

Below are the risks and mitigations associated with the project.

Risk	Mitigation
Personal laptop getting lost or broken	Utilise a cloud-based platform for hosting code such as `GitHub`, and regularly commit code changes. Store relevant documents and research papers in cloud storage. Utilise lab computers in the department of computer science whilst waiting for a replacement.
Unexpected illness or personal circumstances	Contingency time has been reflected in the Timetable.
Data loss or corruption	Regularly backup data such as test logs / benchmarks to cloud storage such as Google Drive. Additionally, store data on an external HDD or SSD.
Time management or overambitious objectives	Regularly produce deliverables and update objectives based on current progress. Discuss with supervisor any proposed changes to the scope of the project.

Table 1: Project risks and their mitigation strategy

6 Legal, Social, Ethical and Professional Issues & Considerations

One professional consideration is utilising proprietary exchange information. This could cause issues with data licensing or the data may have restrictions with how it can be used. If proprietary information is being used, checks will be made to ensure that the data is used legally and within any restrictions stated.

One ethical consideration would be the presence of *personally identifiable information* (PII) in datasets. This raises issues around privacy, consent, and data protection. To mitigate against this, datasets and the corresponding dataset provider will be thoroughly checked to ensure we do not use any dataset containing PII.

There are no further legal, social, ethical or professional concerns with this project.

7 Glossary

Tail latency - the performance experienced by the slowest percentile of tests.

Price-time priority exchange - an algorithmic principle in exchanges that matches orders with the most favourable price and in situations of equal price, the order with the earliest arrival timestamp.

Market events - significant occurrences that can cause price volatility and influence market direction severely in a certain direction.

Spearmen correlation - a value ranging from -1 to 1 indicating how strongly correlated two sets of data points are.

NUMA Architectures - a memory architecture where memory access time is different for all processors in a system, therefore processors access their local memory rather than memory in a remote NUMA node.

Virtual interfaces - a class that declares virtual functions, that must be overriden in any derived class.

Integration tests - form of software testing that verifies different modules or components work together when they are combined.

Personally identifiable information (PII) - any data that can be used to identify an individual. The handling of PII is strictly regulated and the mishandling of it can adhere to penalties and fines.

References

[1] Aras Atalar, Paul Renaud-Goud, and Philippas Tsigas. Analyzing the performance of lock-free data structures: A conflict-based model. In Yoram Moses, editor, *Distributed Computing*, pages 341–355, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. ISBN 978-3-662-48653-5. URL https://link.springer.com/chapter/10.1007/978-3-662-48653-5_23#citeas.

[2] Irina Calciu, Siddhartha Sen, Mahesh Balakrishnan, and Marcos K. Aguilera. Black-box concurrent data structures for numa architectures. In *Proceedings of 22nd ACM International Conference on Architectural Support for*

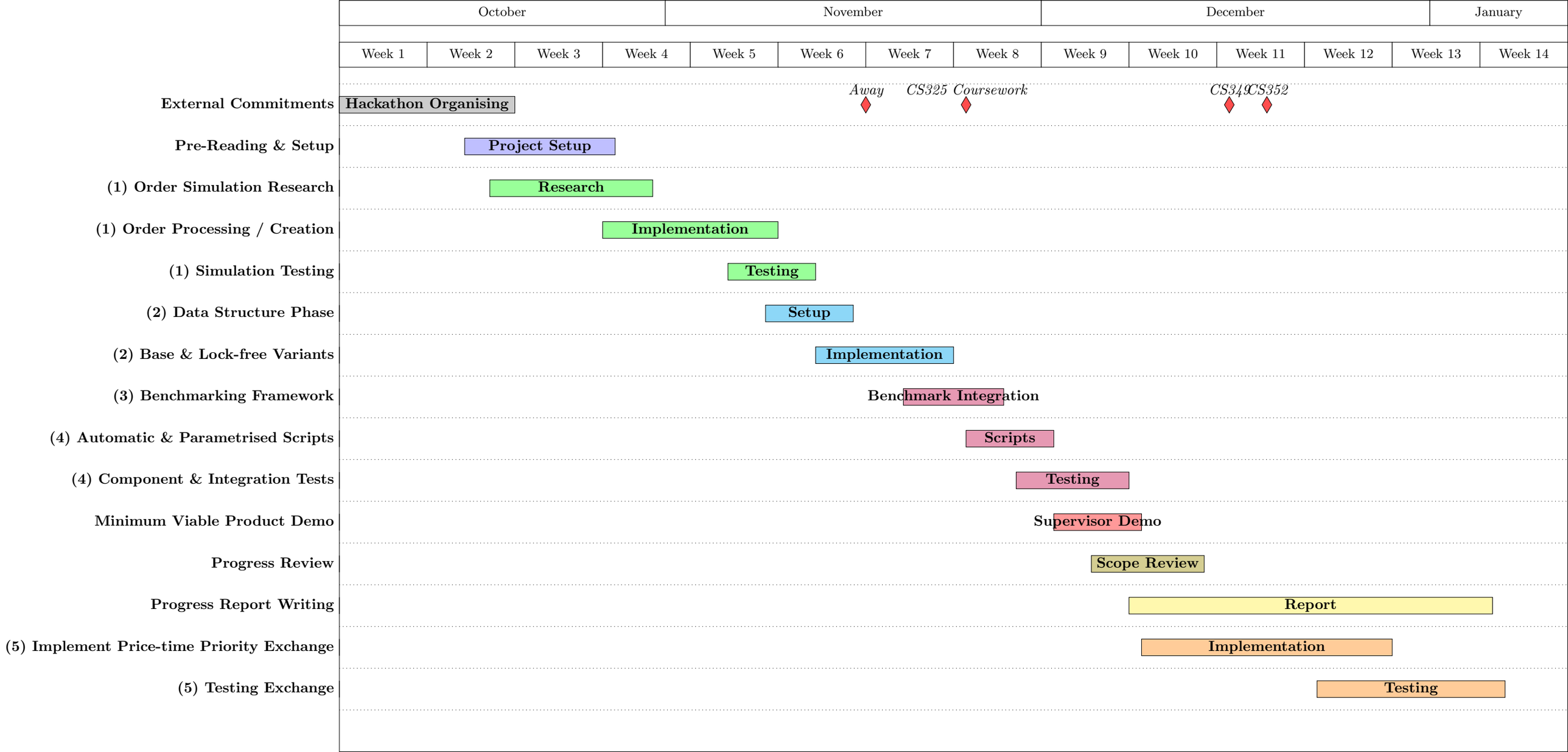
Programming Languages and Operating Systems (ASPLOS), April 2017. URL https://www.microsoft.com/en-us/research/publication/black-box-concurrent-data-structures-numa-architectures-2/. Best Paper Award.

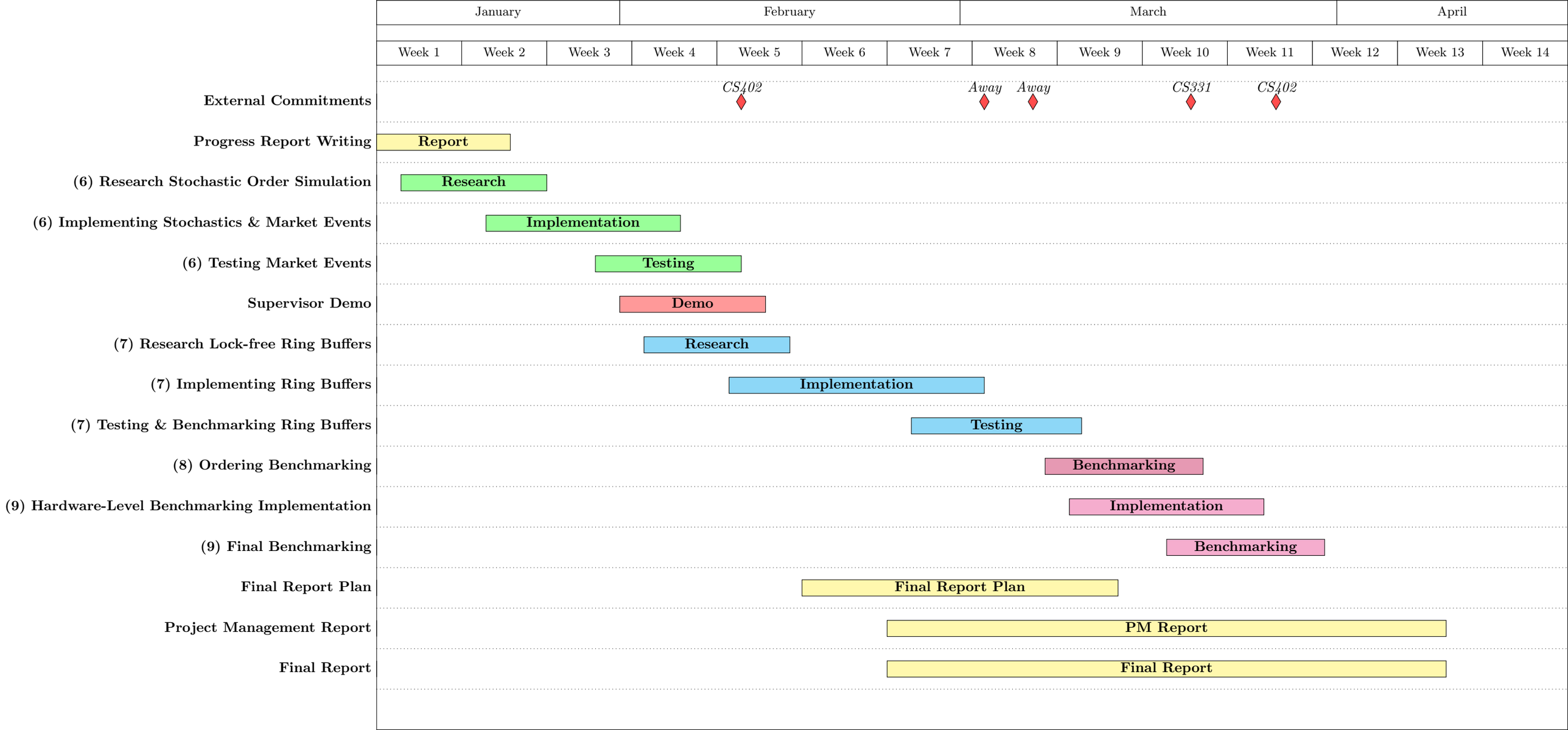
[3] Maged M. Michael. Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI '04)*, pages 35–46. ACM Press, 2004. ISBN 1-58113-807-5. doi: 10.1145/996841.996848. URL https://www.cs.tufts.edu/~nr/cs257/archive/neal-glew/mcrt/Non-blocking%20data%20structures/p35-michael.pdf.

[4] Ruslan Nikolaev. A scalable, portable, and memory-efficient lock-free FIFO queue. *CoRR*, abs/1908.04511, 2019. URL http://arxiv.org/abs/1908.04511.

[5] Ruslan Nikolaev. A scalable, portable, and memory-efficient lock-free fifo queue. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/LIPICS.DISC.2019.28. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2019.28.

[6] Bjarne Stroustrup. *A Tour of C++*. Addison-Wesley Professional, Boston, 3rd edition, 2022. ISBN 978-0-13-681648-5.


October
November
December
January
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8
Week 9
Week 10
Week 11
Week 12
Week 13
Week 14
External Commitments
Hackathon Organising
Away
CS325 Coursework
CS349
CS352
Pre-Reading & Setup
Project Setup
(1) Order Simulation Research
Research
(1) Order Processing / Creation
Implementation
(1) Simulation Testing
Testing
(2) Data Structure Phase
Setup
(2) Base & Lock-free Variants
Implementation
(3) Benchmarking Framework
Benchmark Integration
(4) Automatic & Parametrised Scripts
Scripts
(4) Component & Integration Tests
Testing
Minimum Viable Product Demo
Supervisor Demo
Progress Review
Scope Review
Progress Report Writing
Report
(5) Implement Price-time Priority Exchange
Implementation
(5) Testing Exchange
Testing


January
February
March
April
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8
Week 9
Week 10
Week 11
Week 12
Week 13
Week 14
External Commitments
CS402
Away
Away
CS331
CS402
Progress Report Writing
Report
(6) Research Stochastic Order Simulation
Research
(6) Implementing Stochastics & Market Events
Implementation
(6) Testing Market Events
Testing
Supervisor Demo
Demo
(7) Research Lock-free Ring Buffers
Research
(7) Implementing Ring Buffers
Implementation
(7) Testing & Benchmarking Ring Buffers
Testing
(8) Ordering Benchmarking
Benchmarking
(9) Hardware-Level Benchmarking Implementation
Implementation
(9) Final Benchmarking
Benchmarking
Final Report Plan
Final Report Plan
Project Management Report
PM Report
Final Report
Final Report